# Decision Tree Mechanics and Information Gain

## A Study Report on Entropy, Splitting, and Recursive Tree Learning

*Machine Learning Specialization — Decision Trees Module*

## 1. Placement within the Machine Learning Specialisation

*Welcome to the study of decision trees! This algorithm is one of the most powerful and widely used tools in a developer's machine learning toolbox. While they sometimes receive less attention in academic circles compared to neural networks, decision trees and tree ensembles are frequently the primary choice for winning machine learning competitions and are used in countless real-world applications. They are incredibly effective at handling categorical data and provide a level of interpretability that is highly valuable. Let's dive in and see how they work.*

**Takeaway:** Decision trees are a premier choice for modern applications and competitive machine learning, serving as the fundamental building block for advanced tree ensembles.

## 2. Anatomy of a Decision Tree: The ‘Hanging Plant’ Framework

A decision tree is a model that looks like a hierarchy of choices. If it looks nothing like a biological tree to you, it's okay, don't worry about it! In computer science, we visualize trees starting from the top and growing downwards. You might find it helpful to think of it more like an indoor hanging plant: the roots are at the top, and the leaves fall down toward the bottom.

Using our cat adoption centre example — where we want to classify if an animal is a “Cat” or “Not Cat” — we define the structural components as:

- **Root Node:** The very top node of the tree where the process begins (e.g., Ear Shape).
- **Decision Nodes:** These are the internal oval nodes. They look at a particular feature and, based on its value, decide whether you should go left or right down the tree.
- **Leaf Nodes:** These are the rectangular boxes at the bottom. They don't split any further; they are where the algorithm makes its final prediction.

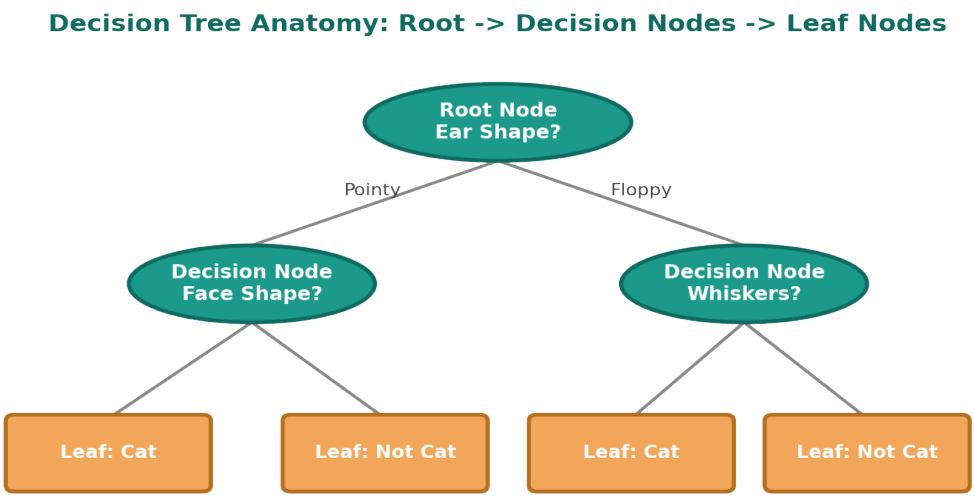



*Figure 1. Root, decision, and leaf nodes for the cat-classification example.*

In our foundational dataset, we use the following features:

| Feature | Categorical Values |
|---|---|
| Ear Shape | Pointy, Floppy |
| Face Shape | Round, Not Round |
| Whiskers | Present, Absent |

## 3. Foundational Mechanics vs Supplementary Context

As we study this, you might feel that the algorithm seems a bit complicated or even “messy.” If it does, reassure yourself that it feels that way to many experts too! This is because the algorithm evolved over decades, with different researchers adding various refinements to make it more effective.

### Foundational Concepts

To get a decision tree to work, you only need to master these core logical requirements:

- **Entropy:** The measure of how “impure” or mixed a set of examples is.
- **Information Gain:** The mathematical logic used to decide which feature to split on at each node.
- **One-Hot Encoding:** A technique to handle features that have more than two possible values.

### Supplementary Context

The “messiness” often comes from historical refinements, such as different ways to measure impurity (like the Gini criteria) or various rules for when to stop growing the tree. While these are useful for preventing overfitting, the core of the algorithm remains the same: a recursive process aimed at maximizing purity.

## 4. Measuring Impurity: The Entropy Formula

To build a tree, we need a way to quantify how “shuffled” our data is. We call this Entropy (H). Let's ground this in our cat dataset: imagine we start at the root node with 10 examples — 5 cats and 5 dogs.

We define $p_1$ as the fraction of positive examples (cats). In this root node, $p_1 = 5/10 = 0.5$.

**The formula for Entropy is:**

$$H(p_1) = -p_1 \log_2(p_1) - (1-p_1) \log_2(1-p_1)$$

$H(p_1) = -p_1 \log_2(p_1) - p_2 \log_2(p_2)$

### Important Conventions

- **Base-2 Logarithm:** We use $\log_2$ specifically so that the peak of our curve is a nice, round 1.0.
- **Handling Zero:** Log(0) is technically undefined, but for entropy, we use the convention that 0 log(0) = 0. This ensures the formula works when a node is perfectly pure.

### The Entropy Curve Behavior

- $p_1 = 0.5 \Rightarrow$ Entropy of 1: This is the highest impurity (a perfect 50/50 mix).
- $p_1 = 0$ or $1 \Rightarrow$ Entropy of 0: This is maximum purity (all examples are either 100% cats or 100% dogs).
- **General Rule:** As $p_1$ moves away from 0.5 and toward 0 or 1, the entropy decreases, meaning the data is becoming more pure.

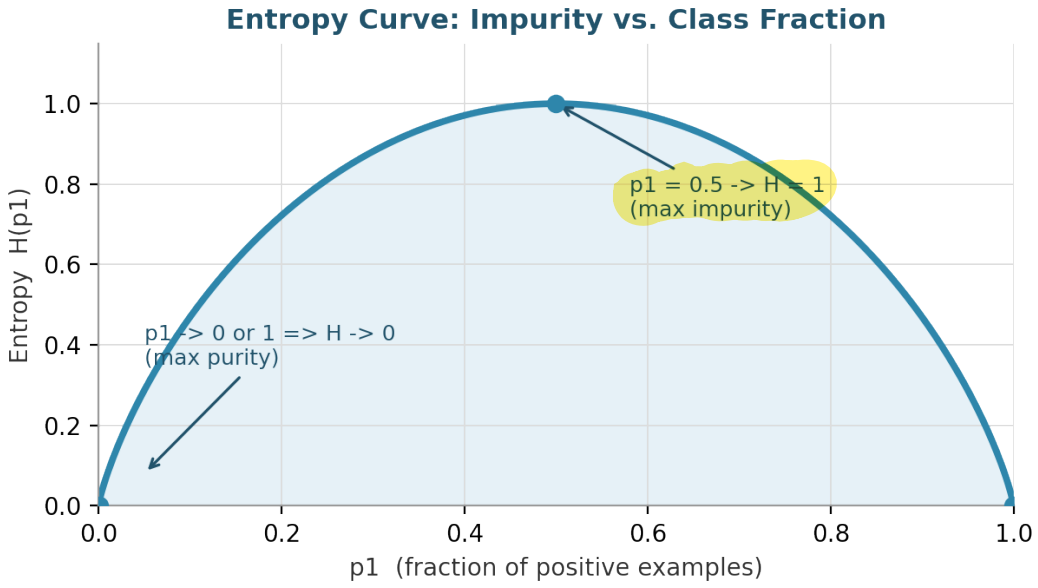



*Figure 2. Entropy peaks at p1 = 0.5 and falls to zero as a node becomes pure.*

## 5. Information Gain (IG) and the Weighted Average

Information Gain is simply the reduction in entropy we get from splitting a node. When we decide which feature to use at a node (Ear Shape vs. Face Shape), we calculate the IG for each and pick the one that gives us the highest value.

### The Role of the Weighted Average

When we split the data into a left branch and a right branch, we need to calculate the average entropy of these new subsets. However, we must use a weighted average.

If one branch has 7 examples and the other has only 3, the branch with 7 examples is more important to our overall impurity score. We define $w^{left}$ and $w^{right}$ as the fraction of total examples that went into each branch (e.g., 7/10 and 3/10).

**The Information Gain formula is:**
**$IG = H(p_1^{root}) - (w^{left}H(p_1^{left}) + w^{right}H(p_1^{right}))$**

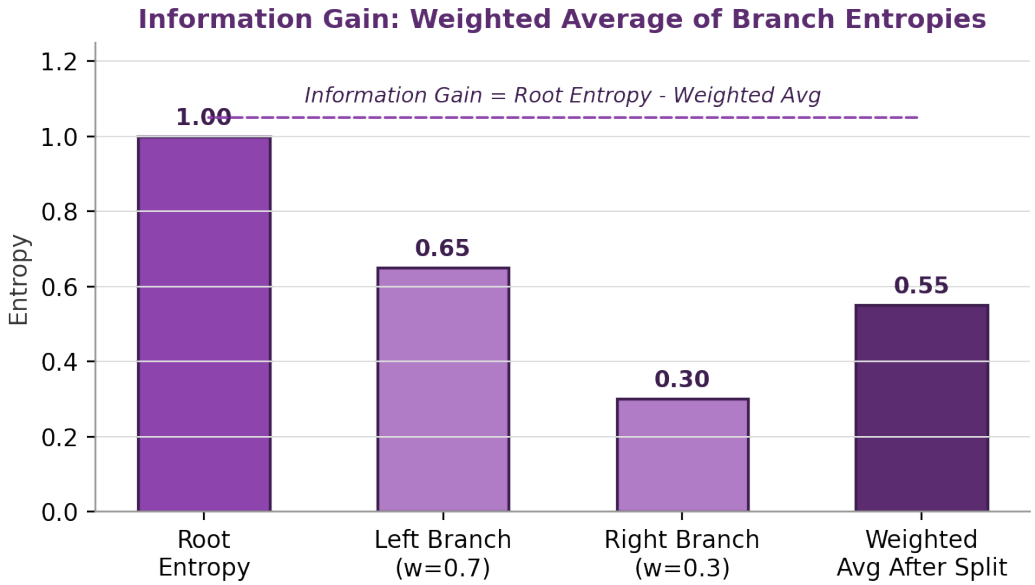



*Figure 3. Illustrative weighted-average entropy calculation feeding into Information Gain.*

**Takeaway:** We always aim to MAXIMIZE Information Gain. A higher IG means a larger reduction in entropy, which brings us closer to the "pure" subsets we need to make accurate predictions.

## 6. The Recursive Learning Algorithm

Building a decision tree is a recursive process. In computer science, recursion refers to "writing code that calls itself." In our case, it means we build the big tree by building smaller and smaller sub-decision trees on subsets of the data.

### Step-by-Step Logic

1. Start with all training examples at the root node.

2. Calculate Information Gain for all available features.
3. Choose the feature with the highest Information Gain and split the data.
4. Create left and right branches and send the corresponding examples down those branches.
5. Repeat: Treat each branch as a new “root” for its specific subset of data and repeat the process.
6. Stop: Continue until you meet your stopping criteria.

### Stopping Criteria

| Criterion | Trigger Condition |
|---|---|
| Perfect Purity | Node is 100% one class (Entropy = 0) |
| Maximum Depth | Tree reaches a set number of “hops” from the root |
| IG Threshold | Reduction in entropy from a split falls below a minimum |
| Example Threshold | Number of examples in a node falls below a minimum |

## 7. Handling Multi-Value Categorical Features

Sometimes, a feature has more than two values. For example, what if Ear Shape could be Pointy, Floppy, or Oval?

To handle this, we use One-Hot Encoding. Instead of one “Ear Shape” feature with three values, we create three separate binary features:

1. “Pointy Ears?” (1 if yes, 0 if no)
2. “Floppy Ears?” (1 if yes, 0 if no)
3. “Oval Ears?” (1 if yes, 0 if no)

* we have N values of a feature
we divide this into N feature
[one hot encoded]

In any given example, only one of these will be “1” (the “hot” value). Just for fun, it's worth noting that this technique is universal — you can use One-Hot Encoding to prepare data for neural networks and logistic regression too!

## 8. Next Steps: Concrete Study Tasks

To consolidate these concepts, try to complete the following checklist. Don't worry if it takes a moment to click; these are the steps that will make you an expert.

- **Manual Derivation:** Take the “Whiskers” feature from the cat dataset and manually calculate its Information Gain using the entropy formula.
- **Implementation:** Practice “One-Hot Encoding” by taking a categorical feature (like “Face Shape”) and converting it into a list of 1s and 0s.

- **Hyperparameter Tuning:** Check the documentation of an open-source ML library (like Scikit-Learn) to see what the default “maximum depth” is. Think about how changing this would affect overfitting.

**Takeaway:** Decision trees reduce to three foundational ideas — entropy, information gain, and one-hot encoding — wrapped in a simple recursive splitting loop. Master the weighted-average IG calculation by hand and the rest of the algorithm follows naturally.